Writing the Design Portion of a Feature Spec

A student guide for drafting the **design / plan** half of a feature specification: how the product will be shaped in data ownership, APIs, screens, tests, and handoff — after the requirements portion is solid.

Write requirements first: writing-feature-requirements.md (header, stories, FRs, initial data model, Gherkin AC).
Full process: framework.md.
Canonical example: feature-2-todo-list-management.md (rich API + Screen) · feature-1-user-auth.md (Test Coverage Map + DoD).

This portion answers: *How do we structure ownership, contracts, UI, and verification so implementers (and Cursor) can build without guessing?*

When to write it: after stories, FRs, Key Entities / initial Data Model, and Gherkin exist. Refine the data model here if APIs/screens force clearer fields. Set `Status: Ready` only when **both** requirements and design sections are complete and consistent.

What "design" means in this kit

Requirements portion	Design portion (this guide)
Who / what / why / proof examples	How the slice is shaped for build
Stories, FRs, SC, Edge Cases, Gherkin	Ownership, API, Screens, test map, DoD
Initial Key Entities + table sketch	Refined Data Model + associations
Product language	Still product language — plus concrete routes, payloads, views

Design here is **not** a separate Figma-only doc and **not** a code dump. Optional UI exports go in `docs/ui/feature-N/` and link from Screen Requirements; the markdown remains the source of truth.

Sections this guide covers (typical order)

Section	Purpose
Data Ownership & Isolation	Multi-user read/write boundaries
API Requirements	Endpoints, auth, payloads, status codes
Screen Requirements	Routes, views, labels, dialogs, empty/error states
Key Entities / Data Model (refine)	Tables, fields, associations aligned to API/UI
Test Coverage Map	Every Gherkin scenario → test file / `it` name
Agent implementation request	Copy-paste Cursor prompt
Definition of Done	Merge checklist for this feature
Out of Scope	Explicit deferrals (with links)
Delivered to Feature X (optional)	Handoff notes for a later feature

Gherkin AC usually already exists from the requirements pass — keep it in sync when you change API or screen labels.

Principles for the design portion

1. **Trace every design choice to a story, FR, or AC.** If nothing authorizes it, cut it or update requirements first.
2. **Be concrete enough to implement; stay out of file-level code.** Name routes, fields, button labels — not Sequelize method bodies.
3. **Contracts before chrome.** Lock API + ownership before polishing Screen Requirements (UI must match the API).
4. **One feature's delta only.** Document what *this* feature adds or changes; point at `features/reference/` for already-shipped baseline.
5. **Match Gherkin strings.** Button labels, error messages, and status codes in Screens/API must match AC quotes.
6. **Design for tests.** The Test Coverage Map should be fillable from your AC without inventing new scenarios.
7. **Say what is *not* included.** Out of Scope prevents scope creep into the next feature.
8. **Follow stack rules by reference.** Point at `.cursor/rules/` (e.g. `ui-style-system.mdc`, `security.mdc`) instead of re-teaching the whole stack in every feature.

Suggested draft order: Ownership → API → Screens → refine Data Model → Test Coverage Map → Agent request → DoD → Out of Scope.

---

1. Data Ownership & Isolation

Required when the feature touches multi-user data (almost always after Feature 1).

What it is

Rules for **who can read or write which rows**. Prevents "it works for me" bugs and matches this kit's security model (`404` for cross-user access — never confirm another user's resource with `403`).

Template

```
## Data Ownership & Isolation

Each user owns their <resources> exclusively. …

| Rule | Requirement |
|------|-------------|
| **Read scope** | … only rows where `userId = req.user.id` |
| **Write scope** | … only when row matches `id` and `req.user.id` |
| **Create scope** | New rows owned by the authenticated user |
| **Cross-user access** | Another user's resource → `404` (not `403`) |
| **UI scope** | UI shows only data returned for the signed-in user |
| **Implementation** | Shared helper in `app/authorization/` — do not duplicate scope in every controller |
```

Principles

Do	Don't
State read, write, create, and cross-user behavior	"Users can only see their own stuff" with no HTTP rule
Align with ADR-0002 / security rules	Invent `403` for "not owned" when the kit standard is `404`
Mention UI scope (what the SPA may show)	Assume the frontend will "just filter" unsafe API data

Example (Feature 2): lists are private; `GET /todo/lists` returns only the caller's rows; wrong-owner `PUT` / `DELETE` → `404`.

---

2. API Requirements

What it is

The HTTP contract for this feature: methods, paths, auth, bodies, success/error shapes. Implementers and backend tests treat this as law.

Template

````
## API Requirements

| Method | Endpoint | Auth | Purpose |
|--------|----------|------|---------|
| `GET` | `/todo/…` | Yes | … |
| `POST` | `/todo/…` | Yes | … |

**Create request body:**
```json
{ "name": "Groceries" }
````

Success response (`200` / `201`):

```
{ "id": 1, "name": "Groceries", "userId": 42 }
```

Error response: `{ "message": "Human-readable explanation." }`
Not found / not owned: `404` (do not use `403`).

```
### Principles

1. **List every endpoint this feature introduces or changes** — not the whole app API.
2. **Mark Auth Yes/No** for each row; protected routes assume Feature 1 session/`Bearer` token.
3. **Show request and response JSON** for create/update (and errors that AC quotes).
4. **Use this app's mount prefix** (`/todo/…` in the Todo example). Stay consistent with existing
routes in `features/reference/api.md`.
5. **Flat JSON** — no `{ success, data }` envelope (per API conventions).
6. **Status codes must match AC** (`201` create, `400` validation, `401` unauthenticated, `404`
missing/not owned).
7. **Do not invent query params or fields** that no FR/AC mentions.

**Delta features:** if you only add a field (e.g. Feature 5 `dueDate`), document the changed
endpoints and the new field — not a rewrite of all todo routes.

---

## 3. Screen Requirements
```
```

````
### What it is

What the user sees and clicks: routes, views, primary actions, dialogs, empty/loading/error states,
and app chrome. Frontend tests and Vitest labels come from here.

### Template

```markdown
## Screen Requirements

### [View: <Name>] — route name `<routeName>`
*   Heading / purpose
*   Primary action: **Label** (`oc-cta` when it is a primary labeled CTA)
*   Fields, dialogs, icon actions with `aria-label`s
*   **Empty state:** exact copy in quotes
*   **Loading / error:** skeleton/progress; `<v-alert type="error">` for failures

**App chrome** (if this feature adds or changes it)
*   e.g. `MenuBar` contents; which routes hide it
````

Principles

1. **Name the view and route** (`home` , `login` , `register`) so router work is unambiguous.
2. **Quote labels that AC/Gherkin use** (`+ New List` , `Sign in` , empty-state sentences).
3. **Specify empty, loading, and error states** — not only the happy layout.
4. **Follow ui-style-system.mdc** — e.g. `oc-cta` on primary labeled CTAs; no labeled buttons stuffed in `v-card-title` .
5. **Icon-only actions need accessible names** (`aria-label` : **Edit list**, **Delete list**).
6. **Call out deferred UI** in parentheses or Out of Scope (*Feature 3 adds Items icon — not in Feature 2*).
7. **Optional:** link Figma/export frames under `docs/ui/feature-N/` — images guide; markdown decides.

Example (Feature 2): single `Dashboard.vue` / `home` ; **My Lists**; **+ New List** dialog; edit/delete dialogs; empty **"No lists yet. Create your first list."**; introduce `MenuBar` .

---

4. Refine Key Entities & Data Model Requirements

You should already have an initial model from the requirements guide. In the design pass:

- Add/adjust columns forced by API payloads or screens.
- Document **associations** (`User hasMany List` , `List belongsTo User`).
- Confirm FK ownership fields match Data Ownership rules.
- For deltas, document **only changed fields** and point at `features/reference/data-model.md` .

Principles

Do	Don't
Keep Field / Type / Rules tables	Paste Sequelize model source code
Align JSON property names with column intent	Silent rename between API and DB without saying so
Note timestamp fields if the API returns them	Invent indexes/performance tuning unless an FR requires it

---
````

5. Test Coverage Map

What it is

The index from **every Gherkin scenario** to at least one automated test. Filling this is part of design for verification — start the map when AC is stable; complete paths/names as you implement (or assign intended files up front).

Template

````
## Test Coverage Map

Each scenario above must map to at least one automated test.

| Story | Scenario | Test file | Test name |
|-------|----------|-----------|-----------|
| US-N.1 | User creates a new list | `backend/tests/lists.test.js` | `User creates a new list` |
| US-N.1 | User creates a list with an empty name | `frontend/tests/Dashboard.test.js` | `User creates a list with an empty name` |
````

Principles

1. **One row per Scenario** — titles must match Gherkin `#### Scenario:` text exactly (for `it("…")` naming).
2. **Backend vs frontend** — API/ownership → Jest + supertest; UI validation/labels → Vitest. Some scenarios need both.
3. **No placeholder tests** — `expect(true).toBe(true)` is not coverage.
4. **Update the map when you split or rename scenarios.**
5. **Prefer real file paths** used in this repo (`backend/tests/…` , `frontend/tests/…`).

---

6. Agent implementation request

What it is

A copy-paste prompt so Cursor (or a teammate) implements **this** feature on the right branch, follows layer order, maps tests, updates living reference, and refuses extra scope.

Place it **after** Test Coverage Map, **before** Definition of Done.

Template

````
## Agent implementation request

Copy when asking Cursor to implement this feature (`@` this file):

```text
Implement Feature N from @features/feature-N-short-name.md on branch `feature/N-short-name`.

Follow layer order in @features/framework.md (models → routes → backend tests → frontend → frontend tests).
Map every Gherkin scenario in the Test Coverage Map; run `npm test` before finishing.
If API routes, payloads, schema, or product rules changed per this spec, update
@features/reference/api.md, @features/reference/data-model.md, and/or
@features/reference/behavior.md in the same PR to match shipped code.
````
````

```
Complete Definition of Done and the merge checklist in @features/framework.md.
Do not implement behavior not in this spec.
```

Reference updates for this feature: `features/reference/…` (list only files this feature will change)

```
### Principles

- Customize Feature ID, `@` path, and branch name.
- List the reference files this feature will touch; omit the reference sentence only if API, schema, and product rules are unchanged.
- Never authorize work that is in **Out of Scope**.

---

## 7. Definition of Done

### What it is

The feature-local merge checklist. If a box is unchecked, the feature is not ready to merge to `dev`.

### Template

```markdown
## Definition of Done

*   [ ] Backend and frontend implemented per this spec (**FR-00N** satisfied)
*   [ ] **Success Criteria (SC-00N)** met
*   [ ] All mapped tests pass (`npm test`)
*   [ ] Test Coverage Map complete
*   [ ] `features/reference/data-model.md` updated (if schema changed)
*   [ ] `features/reference/api.md` updated (if API changed)
*   [ ] `features/reference/behavior.md` updated (if product rules changed)
```

Principles

- Keep DoD aligned with framework.md merge checklist.
- Reference updates are **required when the integrated product changed** — not optional cleanup.
- Do not delete checklist rows to "look done."

---

8. Out of Scope

What it is

Explicit **non-goals** for this feature so readers do not assume missing pieces are accidental. Link forward to the feature that will own deferred work when you know it.

Template

```
## Out of Scope
```
```

```
*   Password reset
*   Full todo dashboard ([Feature 2](./feature-2-todo-list-management.md))
*   Todo items inside lists ([Feature 3](./feature-3-todo-list-item-management.md))
```

Principles

Do	Don't
Name deferred capabilities clearly	Leave silent gaps that look like incomplete AC
Link to later feature files when they exist	Hide in-scope P1 work in Out of Scope
Keep the list short and honest	Dump the entire product roadmap

---

9. Delivered to Feature X (optional)

Use when this feature leaves a temporary placeholder another feature will replace (e.g. Feature 1's minimal home → Feature 2 dashboard).

```
## Delivered to Feature 2

*   Home page is a placeholder only; Feature 2 replaces it with the lists dashboard and MenuBar.
```

Design consistency checklist (before `Status: Ready`)

- [ ] Data Ownership covers read/write/create/cross-user/UI (if multi-user data)
- [ ] API table matches Gherkin methods, paths, status codes, and messages
- [ ] Screen labels and empty/error copy match AC quotes
- [ ] Data model fields support API payloads and screens; associations documented
- [ ] Test Coverage Map has one row per Gherkin scenario (names match)
- [ ] Agent implementation request has correct Feature ID, branch, and reference files
- [ ] Definition of Done present (including reference update lines)
- [ ] Out of Scope lists real deferrals with links where possible
- [ ] Nothing in design contradicts FRs or stories (update requirements first if needed)
- [ ] Catalog row exists / will exist in features/README.md

How the two guides fit together

```
writing-feature-requirements.md          writing-feature-design.md (this file)
─────────────────────────────────        ──────────────────────────────────────
Header, naming                           Data Ownership & Isolation
User stories                             API Requirements
FRs, Assumptions, Edge Cases, SC         Screen Requirements
Key Entities + initial Data Model        Refine Data Model + associations
Gherkin AC                               Test Coverage Map
                                         Agent implementation request
                                         Definition of Done
```

```
                                    Out of Scope (+ optional handoff)

After implementation, update living reference:
  features/reference/writing-living-reference.md
```

End-to-end draft order: requirements guide → this design guide → `Status: Ready` → implement on `feature/N-short-name` using the Agent implementation request.

When unsure, follow framework.md and mirror Features 1–2 in this repo.